INDIAN INSTITUTE OF INFORMATION TECHNOLOGY

ELECTRONICS AND COMMUNICATION ENGINEERING

PATTERN RECOGNITION

PROJECT REPORT

Comparative Analysis of Classical Pattern Recognition Methods Using HOG and PCA Feature Extraction for Fashion-MNIST Classification

Project Category: Image Classification using Classical Machine Learning Techniques

Dataset: Fashion-MNIST | Language: Python | Environment: Google Colab

TEAM PROJECT – SUBMITTED JOINTLY BY

Team Member	Roll Number
RAJESH TURUBILLI	S20240020338
NALLA VAMSI KRISHNA CHOUDHURY	S20240020313

Academic Year: M2026

1. INTRODUCTION

1.1 Importance of Image Classification:

Image classification, assigning a label to a whole image, underlies applications such as product categorisation in online retail, automated inventory management, visual search and recommendation, quality inspection, medical screening and content moderation. In fashion e-commerce, for example, automatically recognising whether a catalogue photograph shows a shirt, a coat or a sandal reduces manual labelling effort and improves search.

1.2 Machine-Learning-Based Image Recognition:

Machine-learning-based image recognition learns the mapping from images to labels from labelled examples. Two broad families exist. In the classical approach, which this project follows, a human-designed feature extractor (such as HOG) is combined with a trainable classifier (such as an SVM or KNN), optionally with dimensionality reduction (such as PCA). In the deep learning approach, convolutional neural networks learn features and classifier jointly. Classical pipelines remain valuable because they are fast, need less data and computation, are easier to interpret, and make the role of each stage explicit, which is ideal for a Pattern Recognition course.

1.3 Problem Statement:

Given a 28 × 28 grayscale image of a clothing item, assign it to one of ten categories. The problem addressed by this project is to determine which combination of feature representation (raw pixels, HOG), dimensionality reduction (none, PCA with 100 components) and classifier (KNN, Linear SVM, RBF SVM) gives the best and most stable classification performance, and to quantify the trade-off between accuracy and running time.

1.4 Objectives:

- To study and implement classical pattern recognition pipelines for Fashion-MNIST classification.
- To extract HOG shape features and apply PCA for dimensionality reduction.
- To implement and compare six classification approaches under identical experimental conditions.
- To evaluate all methods with stratified 3-fold cross-validation using accuracy, macro precision, macro recall and Macro-F1.
- To select the best model by mean cross-validation Macro-F1 and evaluate it once on the official test set.
- To analyse per-class behaviour with the classification report and confusion matrix.

2. DATASET DESCRIPTION:

2.1 Overview of Fashion-MNIST:

Fashion-MNIST is a dataset of article images from an online fashion retailer, released by Xiao, Rasul and Vollgraf (2017). Each sample is a 28 × 28 grayscale image of one clothing item centred on a plain background, with a label from one of ten classes. The notebook loads the dataset with tf.keras.datasets.fashion_mnist.load_data(); the shapes printed by the notebook were (60000, 28, 28) for training images, (60000,) for training labels, (10000, 28, 28) for test images and (10000,) for test labels, with 10 unique classes.

Table 1. Fashion-MNIST dataset specification

Property	Value
Training instances	60,000
Testing instances	10,000
Total instances	70,000
Image dimensions	28 × 28 pixels (grayscale)
Attributes per instance	784 pixel values (28 × 28)
Number of classes	10
Training images per class	6,000
Pixel range (raw)	0 – 255 (integer intensity)

2.2 The Ten Clothing Categories:

Table 2. Fashion-MNIST class labels and descriptions

Label	Class	Description
0	T-shirt/top	Short-sleeved upper-body garment with a round neck; broad, boxy outline.
1	Trouser	Two-legged lower-body garment; tall and narrow with a clear gap between legs.
2	Pullover	Long-sleeved knitted or sweatshirt-type upper-body garment, usually without a front opening.
3	Dress	One-piece garment covering torso and legs; narrow or flared silhouette.
4	Coat	Long-sleeved outer garment, often with a front opening and collar.
5	Sandal	Open footwear with straps; mostly empty (dark) regions.
6	Shirt	Collared, often button-front upper-body garment; visually close to T-shirt/top, Pullover and Coat.
7	Sneaker	Closed low-cut sports shoe with a flat sole.
8	Bag	Handbag or tote of varying shapes, often with handles.
9	Ankle boot	High-top boot covering the ankle.

2.3 Labels, Pixels and Numerical Representation:

A **label** is the integer class index (0–9) that identifies the true clothing category of an image; it is the target the classifier must predict, and the notebook maps indices to names through the class_names list. A **pixel** is one element of the 28 × 28 grid and stores the brightness of that point. Each pixel intensity is an integer from 0 (black) to 255 (white); in Fashion-MNIST the garment is typically bright on a dark background.

2.3.1 Dataset visualisations from the notebook:

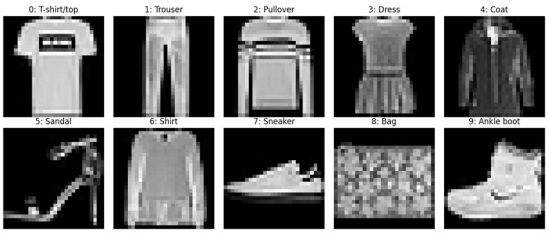


Figure 1. One sample image from each of the ten classes (output of the notebook)

We also tabulated the class counts of the training set: each of the ten classes contains exactly 6,000 images, as shown in Figure 3, so the dataset is perfectly balanced and accuracy is not distorted by class imbalance.



Figure 1. Fashion-MNIST training-set class distribution (output of the notebook)

3. DATA PREPROCESSING:

We performed following preprocessing steps, in this order.

3.1 Loading the Dataset:

The training and test sets were loaded with TensorFlow/Keras into the arrays X_train_img, y_train, X_test_img and y_test. The original 28 × 28 images are retained (X_train_img, X_test_img) because HOG extraction needs the two-dimensional image structure.

3.2 Reshaping Images into Feature Vectors

For the raw-pixel methods, each 28 × 28 image was flattened into a 784-element vector using reshape(len(images), −1). The notebook confirmed the shapes (60000, 784) for the full training matrix and (10000, 784) for the test matrix.

3.3 Pixel Normalisation:

Pixel values were converted to float32 and divided by 255.0, which maps intensities from [0, 255] to [0, 1]. Normalisation keeps feature scales small and consistent, which is important for distance-based methods such as KNN and for the numerical stability and convergence of SVM solvers (especially the RBF kernel, whose gamma = "scale" setting depends on the feature variance).

3.4 Training and Testing Separation:

The official Fashion-MNIST split was respected. All model development, i.e., feature extraction fitting, model comparison and cross-validation, used only training images. The 10,000 official test images were not used for model selection and were used only once, for the final evaluation of the selected model.

3.5 Stratified Sampling of a 20,000-Image Development Set

To keep the six-method cross-validation experiment computationally feasible on Google Colab, a development subset of 20,000 images was drawn from the 60,000 training images with train_test_split(train_size = 20000, stratify = y_train, random_state = 42). Stratification preserves class proportions; the notebook confirmed the class distribution of the development set to be exactly 2,000 images per class.

Table 3. Data splits used in the experiments

Subset	Size	Use
Full official training set	60,000	Source of the development sample
Development set (stratified sample)	20,000 (2,000 per class)	Stratified 3-fold cross-validation, model selection, final training
Official test set	10,000	Final evaluation of the selected model only

4. FEATURE EXTRACTION USING HOG:

4.1 Meaning and Need for Feature Extraction:

Feature extraction transforms raw data into a more informative and compact set of measurements (features) that emphasise the characteristics useful for discriminating classes. Raw pixels are fragile: a shifted, slightly rotated or differently lit garment changes many pixel values even though its category is unchanged, and neighbouring pixels are highly redundant. A good feature representation captures what an object looks like (its edges and shape) rather than the exact brightness at each position, which makes the classification problem easier and the classifier more robust.

4.2 How HOG Works:

The Histogram of Oriented Gradients descriptor, introduced by Dalal and Triggs (2005) for pedestrian detection, describes local object appearance and shape by the distribution of intensity gradient directions. It proceeds in four steps.

Step 1 – Gradient calculation. For every pixel the horizontal and vertical intensity differences (Gx, Gy) are computed with simple derivative filters. The gradient magnitude is $\sqrt{(Gx^2 + Gy^2)}$ and the orientation is atan2(Gy, Gx). Large magnitudes occur at edges, and the orientation gives the edge direction.

Step 2 – Cells and orientation histograms. The image is divided into small cells (4 × 4 pixels in this project). Within each cell, the gradients vote into an orientation histogram (9 bins here), each vote weighted by gradient magnitude. A cell histogram summarises the dominant edge directions in that small region.

Step 3 – Blocks and normalisation. Neighbouring cells are grouped into overlapping blocks (2 × 2 cells here) and the concatenated histograms of a block are normalised (L2-Hys: L2 normalisation, clipping of large values and renormalisation). Block normalisation makes the descriptor less sensitive to local contrast and illumination changes.

Step 4 – Feature vector. The normalised block descriptors are concatenated into one long feature vector for the whole image.

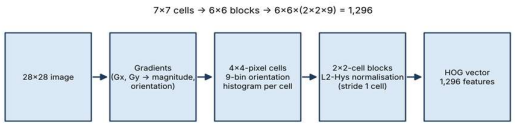


Figure 2. HOG feature extraction pipeline and the feature-vector dimension obtained for a 28 × 28 image

4.3 HOG Parameters Used in the Implementation:

The notebook's extract_hog_features function calls skimage.feature.hog on every image with the following settings.

Table 4. HOG parameters used in the notebook

Parameter	Value	M
orientations	9	Number of orientatio
pixels_per_cell	(4, 4)	Cell size → 7 × 7 = 4
cells_per_block	(2, 2)	Block size → 6 × 6
block_norm	L2-Hys	Block norm
feature_vector	True	Return a flat

4.4 Obtained HOG Feature Dimensions:

With 4 × 4-pixel cells, a 28 × 28 image contains 7 × 7 cells. With 2 × 2-cell blocks moved one cell at a time, there are 6 × 6 = 36 blocks, each contributing 2 × 2 × 9 = 36 values, giving 36 × 36 = 1,296 features per image. This agrees with the shape printed by the notebook: the HOG matrix of the development set is (20,000, 1296), and the HOG matrix of the test set is (10,000, 1296). HOG therefore expands the dimensionality from 784 raw pixels to 1,296 features, but the new features are more informative (edge and shape statistics), which is why PCA is applied afterwards in some pipelines.

4.5 How HOG Represents Shape and Edges:

Clothing categories are largely distinguished by their outlines and internal edges: the two legs of a Trouser, the sleeves of a Pullover, the straps of a Sandal, the handle of a Bag. HOG encodes exactly this information, since edge strength and direction in each cell are captured by the histograms, and the arrangement of cells preserves coarse spatial layout.

4.6 Raw Pixel Features versus HOG Features:

Table 5. Comparison of raw pixel features and HOG features

Aspect	Raw pixels	HOG features
Dimension (this project)	784	1,296
What is encoded	Brightness at each position	Local edge strength and orientation distribution
Robustness to small shifts	Low – a small shift changes many values	Higher – histograms pool gradients over cells
Robustness to lighting/contrast	Low	Higher – block normalisation
Shape information	Implicit only	Explicit
Extraction cost	None	Extra computation per image

5. DIMENSIONALITY REDUCTION USING PCA:

5.1 Dimensionality Reduction and Why PCA Is Used:

Dimensionality reduction maps data from a high-dimensional space to a lower-dimensional one while retaining as much useful information as possible. It reduces storage and computation, can suppress noise and redundancy, and mitigates the curse of dimensionality, in which distances and densities become less reliable as dimension grows. Principal Component Analysis (PCA) is the standard linear, unsupervised technique for this purpose.

5.2 Principal Components and Variance Preservation:

PCA centres the data and computes the eigenvectors of its covariance matrix (equivalently, the singular vectors of the centred data matrix). These eigenvectors are the **principal components**: orthogonal directions ordered by the amount of variance of the data along them. The first component captures the greatest variance, the second the greatest

remaining variance orthogonal to the first, and so on. The fraction of variance explained by component i is λ_i / $\Sigma\lambda_j$, where λ_i is the corresponding eigenvalue.

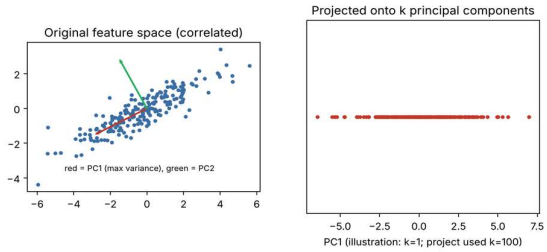


Figure 3. Illustration of PCA: projection onto directions of maximum variance (schematic with synthetic 2-D data)

5.3 How PCA Reduces the Number of Features:

Keeping only the first k principal components, each sample x is replaced by $z = W_k^T(x - \mu)$, where μ is the training mean and W_k contains the first k components. The original d-dimensional vector becomes a k-dimensional vector, discarding the directions of lowest variance, which usually carry mostly noise or redundant detail.

5.4 Choice of 100 Components:

We used n_components = 100 in all PCA pipelines. This is a fixed design choice: 100 components reduce the 784 raw-pixel features by about 87% and the 1,296 HOG features by about 92%, lowering the cost of the SVM stage substantially while retaining the main variance directions. The notebook does not report the explained-variance ratio of the 100 components or test other component counts, so we do not state a retained-variance percentage; testing other values is listed under Future Enhancements.

5.5 PCA in the Experiments

- **On HOG features (Methods 5 and 6):** HOG (1,296-D) → PCA (100-D) → Linear SVM or RBF SVM.
- **On raw pixels (Method 3):** flattened pixels (784-D) → PCA (100-D) → RBF SVM. This is the raw-pixel comparison for the proposed method, isolating the contribution of HOG.

In all cases PCA is a step of a scikit-learn Pipeline, so it is fitted on the training portion of each cross-validation fold only and then applied to the validation fold.

6. CLASSIFICATION ALGORITHMS :

Six classification approaches were implemented in the notebook's build_models() function. Table 6 summarises them; the following sections explain each one.

Table 6. The six classification approaches compared

#	Method	Features	Reduction	Classifier (notebook settings)
1	KNN – Raw Pixels	784 pixels	None	KNN, k = 5, distance-weighted
2	Linear SVM – Raw Pixels	784 pixels	None	LinearSVC, C = 1.0, max_iter = 5000
3	PCA + RBF SVM – Raw Pixels	784 pixels	PCA-100	SVC, RBF, C = 5.0, gamma = 'scale'
4	HOG + Linear SVM	1,296 HOG	None	LinearSVC, C = 1.0, max_iter = 5000
5	HOG + PCA + Linear SVM	1,296 HOG	PCA-100	LinearSVC, C = 1.0, max_iter = 5000
6	HOG + PCA + RBF SVM (proposed)	1,296 HOG	PCA-100	SVC, RBF, C = 5.0, gamma = 'scale'

6.1 Method 1: K-Nearest Neighbors (KNN) on Raw Pixels:

Working principle: KNN is an instance-based method with no explicit training model: a test sample is assigned to the class that dominates among its k nearest training samples under a distance measure (Cover and Hart, 1967). Here k = 5 and neighbours are weighted by the inverse of their distance (weights = "distance"), so closer neighbours count more.

Processing Fashion-MNIST: Each image is a normalised 784-D vector; Euclidean distances to all training vectors are computed and the five closest images vote.

Advantages: Simple, no training phase, naturally non-linear, a strong baseline.

Limitations: Prediction is slow for large training sets, memory-hungry, sensitive to irrelevant dimensions and to shifts of the garment in the image, and Euclidean distance in raw pixel space is not a good measure of shape similarity.

Role in the experiment. Distance-based raw-pixel baseline.

6.2 Method 2: Linear SVM on Raw Pixels:

Working principle: A Support Vector Machine finds the hyperplane that separates classes with the maximum margin (Cortes and Vapnik, 1995). A Linear SVM uses a straight hyperplane $w \cdot x + b = 0$ in the input space. The notebook uses scikit-learn's LinearSVC (C = 1.0, max_iter = 5000), which handles the ten classes with a one-vs-rest scheme.

Processing Fashion-MNIST: The 784-D normalised vector is multiplied by one weight vector per class and the class with the highest score is predicted.

Advantages. Fast prediction, compact model, good for high-dimensional data.

Limitations: Can only draw linear boundaries; raw pixels of clothing classes are not linearly separable, so accuracy is limited.

Role in the experiment. Linear raw-pixel baseline; the weakest reference point.

6.3 Method 3: PCA + RBF SVM on Raw Pixels:

Working principle: Raw pixels are first projected to 100 principal components, then classified by an SVM with the Radial Basis Function kernel $K(x, x') = \exp(-\gamma\|x - x'\|^2)$, which implicitly maps data into a very high-dimensional space where a linear separator corresponds to a non-linear boundary in the original space. The notebook uses C = 5.0 and gamma = "scale"; SVC handles multi-class problems with a one-vs-one scheme.

Processing Fashion-MNIST: 784 pixels → PCA → 100-D vector → kernel SVM.

Advantages: Non-linear boundaries, reduced dimensionality lowers kernel computation cost and may remove noise.

Limitations: Still based on raw intensities, so sensitive to position and illumination; kernel SVM training scales poorly with sample count.

Role in the experiment: The strongest raw-pixel pipeline and the direct comparison for the proposed method, which differs from it only by using HOG features.

6.4 Method 4: HOG + Linear SVM:

Working principle: The 1,296-D HOG descriptor is classified by a LinearSVC (C = 1.0, max_iter = 5000).

Processing Fashion-MNIST: Each image is converted to a HOG vector; the linear SVM learns a weight vector per class over the HOG features.

Advantages: HOG makes classes more linearly separable than raw pixels while keeping the classifier simple and fast to apply.

Limitations: Linear boundary only; 1,296 features without reduction increases training cost compared with PCA-reduced inputs.

Role in the experiment: Isolates the benefit of HOG features with a linear classifier (compare with Method 2).

6.5 Method 5: HOG + PCA + Linear SVM:

Working principle: HOG features are reduced to 100 principal components and classified by LinearSVC (C = 1.0, max_iter = 5000).

Processing Fashion-MNIST: Image → HOG (1,296-D) → PCA (100-D) → linear scores per class.

Advantages: Very compact input and the lowest average execution time in our experiments.

Limitations: PCA discards some discriminative variance and the classifier is linear, which together cost accuracy.

Role in the experiment: Shows the effect of adding PCA to HOG + Linear SVM (compare with Method 4) and the speed-oriented option.

6.6 Method 6: HOG + PCA + RBF SVM (Proposed Method):

Working principle: HOG features are reduced by PCA to 100 components and classified by an RBF-kernel SVM (C = 5.0, gamma = "scale"). This is the combined approach proposed in this project and is described in Chapter 7.

Advantages: Combines shape-aware features, compact decorrelated inputs and a non-linear classifier.

Limitations: Kernel SVMs become expensive for very large training sets; C and gamma were not tuned; the pipeline has more components to configure.

Role in the experiment: The proposed method, evaluated against the five other approaches.

6.7 Linear SVM versus RBF SVM:

Table 7. Linear SVM versus RBF SVM

Aspect	Linear SVM	RBF SVM
Decision boundary	Straight hyperplane in the input space	Curved, non-linear boundary
Kernel	None (linear)	$K(x, x') = \exp(-\gamma\|x - x'\|^2)$
Hyperparameters	C	C and γ
Training cost	Low (LinearSVC)	Higher; grows faster with sample count
Suitable when	Classes are nearly linearly separable	Classes overlap or are curved in feature space
Notebook implementation	LinearSVC, C = 1.0	SVC, kernel = 'rbf', C = 5.0, gamma = 'scale'

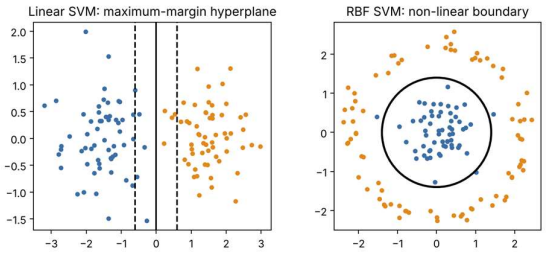


Figure 4. Linear SVM maximum-margin hyperplane versus RBF SVM non-linear boundary (schematic)

7. PROPOSED METHOD: HOG + PCA + RBF SVM:

The proposed approach of this project is the combined pipeline HOG + PCA + RBF SVM, in which three complementary techniques are chained so that each compensates for a limitation of the others.

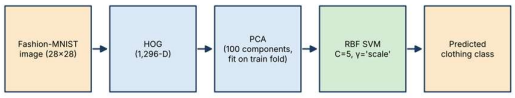


Figure 5. Complete proposed methodology: image → HOG → PCA → RBF SVM → predicted class

7.1 Pipeline Description:

- **Original Fashion-MNIST image:** a 28 × 28 grayscale clothing image.
- **HOG feature extraction:** 9 orientations, 4 × 4 cells, 2 × 2 blocks, L2-Hys → 1,296-D shape descriptor.
- **PCA dimensionality reduction:** 1,296 → 100 principal components, fitted on training data only.
- **RBF SVM classification:** SVC with RBF kernel, C = 5.0, gamma = 'scale'.
- **Predicted clothing category:** one of the ten classes.

7.2 Why the Combination May Improve Performance:

HOG supplies features centred on edges and shape, which are the cues that distinguish garments, and which are more tolerant of small intensity and position changes than raw pixels. PCA compacts the 1,296 correlated HOG features into 100 orthogonal components, reducing redundancy and noise and making the kernel computation cheaper. The RBF-kernel SVM can then draw non-linear boundaries between classes whose HOG-feature distributions overlap or are curved, which a linear classifier cannot. These are the expected mechanisms; whether the combination actually helps is judged by the experiments in Chapter 10, where it achieved the best cross-validation scores of the six methods.

8. EXPERIMENTAL METHODOLOGY:

8.1 Training Procedure:

All experiments were implemented in Python in Google Colab. The random seed was fixed to 42 for NumPy, the stratified sampling, the cross-validation shuffling and the PCA and LinearSVC random states, to make the experiment reproducible. HOG features of the 20,000 development images were computed once and reused for the three HOG-based methods.

8.2 Stratified 3-Fold Cross-Validation:

The 20,000-image development set was divided into three folds with StratifiedKFold(n_splits = 3, shuffle = True, random_state = 42). In each iteration, two folds (about 13,333 images) are used to train the model and the remaining fold (about 6,667 images) is used for validation; every image is used for validation exactly once. Stratification keeps the class proportions equal in every fold.

Cross-validation is used because a single train/validation split gives a result that depends on which images happened to be chosen. Averaging over folds gives a more reliable estimate of generalisation, and the fold-to-fold standard deviation measures stability.

8.3 Identical Folds for All Methods:

A single StratifiedKFold object was created and the same split of indices (cv.split(X_dev, y_dev)) was applied to every method. Methods 1–3 use the raw-pixel matrix and Methods 4–6 use the HOG matrix, but the rows correspond to the same images, so all six methods are trained and validated on exactly the same images in each fold. This makes the comparison fair and paired.

8.4 Preventing Data Leakage:

Data leakage occurs when information from the validation data influences training. PCA learns the data mean and principal directions from the data it is fitted on. In the notebook, PCA is a stage of a scikit-learn Pipeline that is created freshly for every fold (build_models()[name]) and fitted with model.fit(X_train_fold, y_train_fold), so the mean and components are computed from the training fold only; the validation fold is merely transformed. HOG is computed independently for each image and has no learned parameters, so computing it once for the development set does not leak information between folds.

8.5 Model Selection and Final Evaluation:

For each method the fold scores were aggregated into mean and standard deviation, and the models were ranked by mean cross-validation Macro-F1. The top-ranked model (best_model_name = summary.index[0]) was selected. It was then retrained on all 20,000 development images, test HOG features were extracted from the 10,000 official test images, and the model was evaluated once on the official test set. The test set was never used for model selection.

8.6 Experimental Parameters:

Table 8. Experimental parameters used in the notebook

Parameter	Value
Dataset	Fashion-MNIST (tf.keras.datasets)
Development sample	20,000 images, stratified (2,000 per class)
Official test set	10,000 images
Pixel scaling	float32, divided by 255.0
Cross-validation	StratifiedKFold, 3 folds, shuffle = True
Random seed	42
HOG	9 orientations, (4, 4) pixels/cell, (2, 2) cells/block, L2-Hys → 1,296 features
PCA	100 components, random_state = 42
KNN	n_neighbors = 5, weights = 'distance'
Linear SVM	LinearSVC, C = 1.0, max_iter = 5000, random_state = 42
RBF SVM	SVC, kernel = 'rbf', C = 5.0, gamma = 'scale'
Selection criterion	Highest mean CV Macro-F1
Software	TensorFlow 2.21.0, scikit-learn 1.6.1, scikit-image, Google Colab

9. PERFORMANCE EVALUATION METRICS:

For each class, a prediction is a True Positive (TP) if the sample of that class is correctly predicted, a False Positive (FP) if a sample of another class is wrongly predicted as this class, a False Negative (FN) if a sample of this class is predicted as another class, and a True Negative (TN) otherwise.

9.1 Accuracy:

$$Accuracy = (number\ of\ correct\ predictions) / (total\ predictions)$$
$$= (TP + TN) / (TP + TN + FP + FN)$$

Accuracy is the proportion of all images classified correctly.

9.2 Precision:

$$Precision = TP / (TP + FP)$$

Precision answers: of the images predicted as this class, how many really belong to it? Low precision means many false alarms.

9.3 Recall:

$$Recall = TP / (TP + FN)$$

Recall answers: of the images that truly belong to this class, how many were found? Low recall means many items of the class are missed.

9.4 F1-Score:

$$F1 = 2 \times Precision \times Recall / (Precision + Recall)$$

F1 is the harmonic mean of precision and recall; it is high only when both are high.

9.5 Macro-Averaging:

$$Macro\text{-}F1 = (1/K) \Sigma F1_k \quad (k = 1 \ldots K,\ K = 10\ classes)$$

Macro-averaging computes the metric separately for each class and takes the unweighted mean, so every class counts equally regardless of its size. The notebook uses average = "macro" for F1, precision and recall. Because Fashion-MNIST is balanced, macro and weighted averages are nearly identical, but Macro-F1 was chosen as the selection criterion since it penalises weakness on any single class.

9.6 Confusion Matrix:

The confusion matrix is a 10 × 10 table whose entry (i, j) counts test images of true class i predicted as class j. The diagonal holds the correct predictions (true positives for each class) and off-diagonal entries show exactly which classes are confused.

9.7 Mean and Standard Deviation Across Folds:

$$Mean = (1/n) \Sigma s_i ; \quad Std = \sqrt{[\ \Sigma (s_i - Mean)^2 / (n - 1)\ ]} \quad (n = 3\ folds)$$

The mean estimates expected performance; the standard deviation measures how much the score varies from fold to fold, i.e., stability. The values reported in this report are consistent with the sample standard deviation (n − 1), which we verified by recomputing the proposed method's accuracy standard deviation from its three fold scores (≈ 0.0014).

9.8 Why Multiple Metrics Are Needed:

A single number can be misleading. Accuracy does not reveal which classes fail; precision and recall expose different kinds of error; F1 balances them; the confusion matrix identifies specific class pairs that are confused; and the standard deviation reveals instability. Reporting all of them gives a complete and honest picture of classifier behaviour.

10. EXPERIMENTAL RESULTS AND COMPARISON :

Important: this chapter reports **cross-validation** results on the 20,000-image development set. They are different from the **final test-set** results in Chapter 11. Values below were taken from the notebook's printed cross-validation tables (fold-level results and the aggregated comparison table) and agree with the reference values supplied for this project.

10.1 Cross-Validation Comparison Table:

Table 9. Cross-validation comparison of the six methods (mean over 3 folds; ± = standard deviation)

Rank	Method	Mean Accuracy	Acc. Std	Mean Macro-F1	F1 Std	Mean Precision	Mean Recall	Avg. Time (s)
1	**HOG + PCA + RBF SVM (proposed)**	**0.8936**	**0.001**	**0.8933**	**0.0014**	**0.8932**	**0.8936**	**12.04**
2	PCA + RBF SVM – Raw Pixels	0.8816	0.002	0.8812	0.0024	0.8815	0.8816	17.61
3	HOG + Linear SVM	0.8654	0.001	0.8649	0.0013	0.8648	0.8654	20.53
4	HOG + PCA + Linear SVM	0.8613	0.004	0.8598	0.0039	0.8594	0.8613	5.33
5	KNN – Raw Pixels	0.8311	0.003	0.8311	0.0036	0.8353	0.8312	9.64
6	Linear SVM – Raw Pixels	0.8203	0.005	0.8187	0.0052	0.8182	0.8204	27.28

Precision and recall are macro-averaged: Average time is the mean over the three folds of the time for fitting and predicting the validation fold, measured by the notebook (for pipelines it includes the PCA step; one-time HOG extraction time is not included).

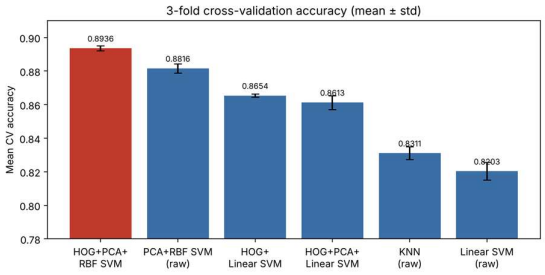


Figure 6. Mean cross-validation accuracy (with standard-deviation error bars) of the six methods

10.2 Per-Fold Results:

Table 10. Fold-level cross-validation results printed by the notebook

Method	Fold	Accuracy	Macro-F1	Time (s)
KNN – Raw Pixels	1	0.8352	0.8350	13.16
	2	0.8308	0.8306	8.81
	3	0.8275	0.8278	6.93
Linear SVM – Raw Pixels	1	0.8232	0.8209	28.00
	2	0.8236	0.8225	27.16
	3	0.8143	0.8128	26.67
PCA + RBF SVM – Raw Pixels	1	0.8796	0.8791	23.45
	2	0.8847	0.8839	17.63
	3	0.8806	0.8806	11.76
HOG + Linear SVM	1	0.8646	0.8641	20.45
	2	0.8652	0.8643	19.91
	3	0.8665	0.8664	21.23
HOG + PCA + Linear SVM	1	0.8655	0.8639	5.14
	2	0.8614	0.8596	5.19
	3	0.8572	0.8560	5.66
HOG + PCA + RBF SVM	1	0.8952	0.8948	13.08
	2	0.8931	0.8925	11.14
	3	0.8926	0.8925	11.91

10.3 Discussion of the Cross-Validation Results:

Highest accuracy and Macro-F1: The proposed HOG + PCA + RBF SVM achieved both the highest mean cross-validation accuracy (0.8936) and the highest mean Macro-F1 (0.8933) and was therefore selected as the best model by the selection criterion.

Improvement over the other methods. The following table gives the difference between the proposed method and each other method, in absolute percentage points of the cross-validation means.

Table 11. Cross-validation improvement of the proposed method over each other method

Compared method	Δ Accuracy (pp)	Δ Macro-F1 (pp)
PCA + RBF SVM – Raw Pixels	+1.20	+1.21
HOG + Linear SVM	+2.82	+2.84
HOG + PCA + Linear SVM	+3.23	+3.35
KNN – Raw Pixels	+6.25	+6.22
Linear SVM – Raw Pixels	+7.33	+7.46

The differences were computed from the table values (pp = percentage points). The 1.2-point gain over PCA + RBF SVM on raw pixels is the contribution of HOG with otherwise identical PCA and classifier settings.

Meaning of the standard deviation: The standard deviation across the three folds shows how stable a method is. The proposed method (0.0014) and HOG + Linear SVM (0.0010) have the smallest accuracy standard deviations, so their scores barely changed from fold to fold, whereas Linear SVM on raw pixels (0.0053) was the least stable. Because the gap between the proposed method and the runner-up (0.0120) is several times larger than the standard deviations, the ranking of the top two methods was consistent across folds (the proposed method led in every fold). We caution, however, that with only three folds and overlapping training sets, the standard deviation is a rough indicator and no formal statistical significance test was performed.

Performance versus execution time: HOG + PCA + Linear SVM was the fastest (5.33 s) but 3.2 points less accurate than the proposed method. KNN (9.64 s) was fast in this measurement but among the least accurate. The proposed method needed 12.04 s on average, less than PCA + RBF SVM on raw pixels (17.61 s), HOG + Linear SVM (20.53 s) and Linear SVM on raw pixels (27.28 s), while providing the highest accuracy, so it offers the best balance of performance and execution time in this experiment. If the lowest possible run time matters most, HOG + PCA + Linear SVM is the more economical choice. Timings come from a shared Google Colab runtime and should be read as indicative.

11. FINAL TEST RESULTS:

The model selected by cross-validation, **HOG + PCA + RBF SVM**, was retrained on the 20,000 development images and evaluated once on the official 10,000-image test set (training and prediction took about 0.4 minutes). The results below are **test-set** results and must not be confused with the cross-validation scores of Chapter 10.

Table 12. Final test-set performance of the selected model (HOG + PCA + RBF SVM)

Metric	Value
Selected model	HOG + PCA + RBF SVM
Test accuracy	0.8940
Test macro precision	0.8942
Test macro recall	0.8940
Test Macro-F1	0.8941
Development (training) size	20,000
Test size	10,000

The test accuracy (0.8940) is very close to the mean cross-validation accuracy of the same model (0.8936), which indicates that the cross-validation estimate was realistic and that the model did not over-fit the development set. Only the selected model was evaluated on the test set, so this report makes **no claim** about how the other five methods would rank on the test set.

11.1 Classification Report:

Table 13. Test-set classification report of HOG + PCA + RBF SVM (output of the notebook)

Class	Precision	Recall	F1-score	Support
T-shirt/top	0.8579	0.8450	0.8514	1000
Trouser	0.9847	0.9670	0.9758	1000
Pullover	0.8342	0.8300	0.8321	1000
Dress	0.8871	0.9040	0.8955	1000
Coat	0.8217	0.8390	0.8303	1000
Sandal	0.9676	0.9560	0.9618	1000
Shirt	0.7126	0.7090	0.7108	1000
Sneaker	0.9309	0.9570	0.9438	1000
Bag	0.9760	0.9770	0.9765	1000
Ankle boot	0.9696	0.9560	0.9627	1000
Accuracy			**0.8940**	**10000**
Macro avg	**0.8942**	**0.8940**	**0.8941**	**10000**
Weighted avg	**0.8942**	**0.8940**	**0.8941**	**10000**

11.2 Interpretation:

- **Best-recognised classes:** Bag (F1 0.9765), Trouser (0.9758), Ankle boot (0.9627) and Sandal (0.9618) all exceed 0.96, followed by Sneaker (0.9438). These categories have distinctive outlines that HOG captures well.
- **Intermediate classes:** Dress (0.8955), T-shirt/top (0.8514), Pullover (0.8321) and Coat (0.8303).

- **Hardest class:** Shirt (precision 0.7126, recall 0.7090, F1 0.7108), far below all others, because it resembles several upper-body garments.
- Precision and recall are similar for every class, so the classifier is not strongly biased toward predicting or avoiding any particular class.

12. CONFUSION MATRIX ANALYSIS :

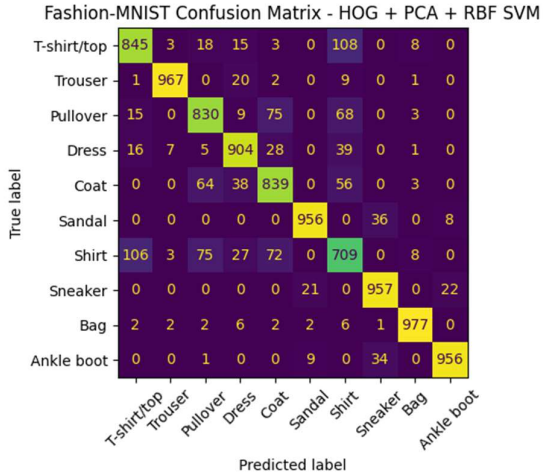


Figure 7. Confusion matrix of HOG + PCA + RBF SVM on the 10,000 official test images (output of the notebook)

12.1 True Positives and Correct Classification

The diagonal gives the true positives: T-shirt/top 845, Trouser 967, Pullover 830, Dress 904, Coat 839, Sandal 956, Shirt 709, Sneaker 957, Bag 977 and Ankle boot 956. In total 8,940 of 10,000 images were classified correctly (accuracy 0.8940) and 1,060 were misclassified. Bag (977), Trouser (967), Sneaker (957), Sandal (956) and Ankle boot (956) are correctly classified for at least 95% of their images.

12.2 Misclassifications and Commonly Confused Classes

Table 14. Largest off-diagonal confusions in the test set

True class → Predicted class	Count
T-shirt/top → Shirt	108
Shirt → T-shirt/top	106
Pullover → Coat	75
Shirt → Pullover	75
Shirt → Coat	72
Pullover → Shirt	68
Coat → Pullover	64
Coat → Shirt	56
Sandal → Sneaker	36
Ankle boot → Sneaker	34

- **Upper-body garments (T-shirt/top, Pullover, Coat, Shirt):** nearly all the errors fall inside this group. T-shirt/top and Shirt are confused in both directions (108 and 106 images), and Shirt is also frequently mistaken for Pullover (75) and Coat (72). Shirt images are scattered across T-shirt/top (106), Pullover (75), Dress (27), Coat (72) and others, which explains its low recall of 0.709. Pullover and Coat are also mixed up (75 and 64) because both are long-sleeved with similar outlines at 28 × 28 resolution.
- **Dress** is mostly confused with Coat (28 → Coat; Coat → Dress 38) and T-shirt/top (16 and 15), and 20 Trouser images were predicted as Dress.
- **Footwear (Sandal, Sneaker, Ankle boot):** the few errors are between neighbouring shoe types: Sandal → Sneaker (36), Ankle boot → Sneaker (34), Sneaker → Ankle boot (22) and Sneaker → Sandal (21). Footwear is almost never confused with clothing or bags.
- **Bag and Trouser** are the cleanest classes, with 23 and 33 total errors respectively.

These patterns are visual explanations rather than something tested with additional experiments: the confused categories are exactly those whose silhouettes are alike, and whose distinguishing details (collar, buttons, texture) are limited at 28 × 28 resolution.

13. DISCUSSION:

13.1 Why HOG Is Useful for Clothing Images:

Clothing categories differ mainly in their outlines and internal edges, and HOG describes exactly these properties through cell-wise gradient orientation histograms with local contrast normalisation. In our experiments, HOG improved the linear SVM from 0.8203 (raw pixels) to 0.8654 (HOG) in mean cross-validation accuracy, an increase of 4.5 points, and, comparing the two PCA + RBF SVM pipelines that differ only in the input features, from 0.8816 (raw pixels) to 0.8936 (HOG).

13.2 Effect of PCA:

The effect of PCA depends on the classifier. With a linear SVM on HOG features, adding PCA slightly reduced mean accuracy (0.8654 → 0.8613) and increased the fold-to-fold variation (0.0010 → 0.0041), but it reduced the average time from 20.53 s to 5.33 s. With the RBF SVM, PCA on raw pixels allowed a strong 0.8816 accuracy; the notebook did not run an RBF SVM on un-reduced features, so we cannot isolate PCA's effect on accuracy for the RBF classifier. In general, our results show PCA to be mainly a means of reducing computational cost while largely preserving accuracy.

13.3 Linear versus RBF SVM:

The RBF SVM variants were clearly the best two methods (0.8936 and 0.8816), while the linear variants were lower (0.8654 and 0.8613 on HOG, and 0.8203 on raw pixels). A plausible explanation is that clothing classes overlap and form curved clusters in feature space, which a single linear hyperplane per class cannot separate, whereas the RBF kernel can. The comparison is, however, not perfectly controlled: the RBF SVM used C = 5.0 while the linear SVM used C = 1.0, and neither was tuned.

13.4 Did the Proposed Method Outperform the Others?:

In cross-validation on the development set, yes: HOG + PCA + RBF SVM had the highest mean accuracy and Macro-F1 of the six methods, with low variance. On the official test set it reached accuracy 0.8940 and Macro-F1 0.8941, in line with its cross-validation estimate. Since only this model was evaluated on the test set, we do not claim it is best on the test set; we can only say that it was the best by cross-validation and that its test result is consistent.

13.5 Limitations of the Experiment:

- Only 20,000 of the 60,000 training images were used for development, so absolute scores may differ with the full training set.
- Hyperparameters (C, gamma, k, HOG settings, PCA components) were fixed, not tuned, and the PCA explained variance was not examined.
- Three-fold cross-validation was used with a single random seed; the std values from three folds are coarse and no statistical significance test was done.
- Execution times were measured on a shared Colab runtime, cover fitting and prediction only, and exclude HOG extraction time, so they are approximate.
- Only the selected model was evaluated on the test set, so a test-set comparison of all methods is not available.
- The explanations of why particular classes are confused are based on the confusion matrix, not on further experiments.

14. CONCLUSION:

We implemented and compared six classical pattern recognition pipelines for Fashion-MNIST classification under an identical, leakage-free evaluation protocol: stratified 3-fold cross-validation on a stratified 20,000-image development set, with PCA fitted inside each training fold. The proposed HOG + PCA + RBF SVM obtained the best cross-validation accuracy (0.8936 ± 0.0014) and Macro-F1 (0.8933 ± 0.0014), ahead of PCA + RBF SVM on raw pixels (0.8816 / 0.8812) and the other four methods, while also running faster than three of the other five. Retrained on the development set and tested once on the 10,000 official test images, it reached an accuracy of 0.8940, macro precision 0.8942, macro recall 0.8940 and Macro-F1 0.8941.

From the results we learned that (1) shape-oriented HOG features are more effective than raw pixels for clothing images, (2) a non-linear RBF kernel is considerably more effective than a linear boundary on this data, (3) PCA greatly reduces cost with a small accuracy effect for linear classifiers, and (4) the confusion matrix is essential for understanding where a classifier fails: the Shirt class, confusable with T-shirt/top, Pullover and Coat, remains the main difficulty, while Bag, Trouser and footwear are recognised reliably.

Overall, the work shows that a carefully designed classical pipeline, feature extraction, dimensionality reduction and a kernel classifier, can reach close to 90% accuracy on Fashion-MNIST with modest computation, and provides a transparent and interpretable baseline for image classification.

15. FUTURE ENHANCEMENTS:

- **Different HOG parameters:** test other cell sizes, block sizes, orientation counts and normalisation schemes.
- **Different PCA component counts:** analyse the explained-variance curve and test, for example, 50, 150 or 200 components.
- **Hyperparameter tuning:** tune C and gamma of the RBF SVM (and C of the linear SVM, k of KNN) with grid or randomised search inside the cross-validation.
- **Use all 60,000 training instances:** train the final model on the complete training set and, if the budget allows, use 5- or 10-fold cross-validation.
- **Additional classical classifiers:** compare Random Forest, Gradient Boosting, Logistic Regression, Naive Bayes and LDA, as well as other descriptors such as LBP.
- **Comparison with CNN-based approaches:** compare the classical pipelines with convolutional neural networks to measure the accuracy gain relative to computational cost and interpretability.
- **Test-set comparison of all methods:** evaluate all six methods on the official test set, together with statistical tests of the differences.

16. REFERENCES:

[1] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: a novel image dataset for benchmarking machine learning algorithms," arXiv preprint arXiv:1708.07747, 2017.

[2] N. Dalal and B. Triggs, "Histograms of oriented gradients for human detection," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), vol. 1, 2005, pp. 886–893.

[3] I. T. Jolliffe, Principal Component Analysis, 2nd ed. New York, NY, USA: Springer, 2002.

[4] H. Hotelling, "Analysis of a complex of statistical variables into principal components," Journal of Educational Psychology, vol. 24, no. 6, pp. 417–441, 1933.

[5] C. Cortes and V. Vapnik, "Support-vector networks," Machine Learning, vol. 20, no. 3, pp. 273–297, 1995.

[6] T. Cover and P. Hart, "Nearest neighbor pattern classification," IEEE Transactions on Information Theory, vol. 13, no. 1, pp. 21–27, 1967.

[7] R.-E. Fan, K.-W. Chang, C.-J. Hsieh, X.-R. Wang, and C.-J. Lin, "LIBLINEAR: a library for large linear classification," Journal of Machine Learning Research, vol. 9, pp. 1871–1874, 2008.

[8] C.-C. Chang and C.-J. Lin, "LIBSVM: a library for support vector machines," ACM Transactions on Intelligent Systems and Technology, vol. 2, no. 3, art. 27, 2011.

[9] F. Pedregosa et al., "Scikit-learn: machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[10] Scikit-learn developers, "scikit-learn: Machine Learning in Python – User Guide and API Documentation," [Online]. Available: https://scikit-learn.org/stable/ (accessed 2026).

[11] S. van der Walt et al., "scikit-image: image processing in Python," PeerJ, vol. 2, art. e453, 2014.

[12] M. Abadi et al., "TensorFlow: large-scale machine learning on heterogeneous systems," 2015. [Online]. Available: https://www.tensorflow.org/

[13] R. O. Duda, P. E. Hart, and D. G. Stork, Pattern Classification, 2nd ed. New York, NY, USA: Wiley, 2001.

17. PROJECT SOURCE CODE AND GITHUB REPOSITORY:

The complete Python source code of this project is provided through a GitHub repository rather than reproducing the entire implementation inside this report. The repository contains the Python implementation (the Google Colab notebook) and the relevant project files listed below.

GitHub Repository Link:

https://github.com/rajesht11-de/PR_PROJECT_M2026_GROUP_9

17.1 Repository Contents:

The following files were produced by the project and should be included in the repository (adjust the list to match what is actually uploaded).

Table 15. Files of the project repository

File	Description
PR_project.ipynb	Google Colab notebook with the complete Python implementation: data loading, preprocessing, HOG extraction, six models, cross-validation, final test evaluation and plots
fashion_mnist_results_bundle.zip	ZIP bundle of the result files below, generated by the notebook
fashion_mnist_cv_fold_results.csv	Fold-level cross-validation results (6 methods × 3 folds)
fashion_mnist_cv_summary.csv	Aggregated cross-validation summary
fashion_mnist_test_predictions.csv	True and predicted labels for the 10,000 test images
fashion_mnist_test_classification_report.csv	Test-set classification report
fashion_mnist_final_metrics.csv	Final test metrics of the selected model
fashion_mnist_experiment_settings.json	Experimental settings (HOG, PCA, seed, selected model)
Fashion_MNIST_Model_Comparison.xlsx	Formatted model-comparison table
PR_Report_FFF (.docx)	This report
PR_presentation_F	presentation

APPENDIX A: INDIVIDUAL CONTRIBUTIONS OF TEAM MEMBERS

This project is a **joint team submission**. Both members planned, executed, reviewed and documented the project together,. The division below lists the principal areas in which each member took the lead; both contributions were equally important to the successful completion of the project.

Table 16. Contribution summary

Team Member	Roll Number
RAJESH TURUBILLI	S20240020338
NALLA VAMSI KRISHNA CHOUDHURY	S20240020313

Member 1: RAJESH TURUBILLI

Roll Number: S20240020338

1. Dataset research and selection:

- Studied the Fashion-MNIST dataset.
- Examined the dataset structure, attributes, labels and ten clothing categories.
- Assessed the dataset's suitability for pattern recognition.

2. Data loading and preprocessing:

- Implemented dataset loading using Python and TensorFlow.
- Performed image reshaping and pixel normalisation.
- Prepared the training, development and testing datasets.
- Implemented stratified sampling.

3. Feature extraction:

- Studied and implemented Histogram of Oriented Gradients (HOG).
- Experimented with HOG parameters.
- Generated the HOG feature representations.

4. Dimensionality reduction:

- Studied and implemented Principal Component Analysis (PCA).
- Applied PCA to reduce the dimensionality of the extracted features.

5. Documentation:

- Contributed to explaining the dataset characteristics, preprocessing, HOG and PCA in the report.

Member 2: NALLA VAMSI KRISHNA CHOUDHURY

Roll Number: S20240020313

1. Classification algorithm implementation:

- Studied and implemented K-Nearest Neighbors (KNN).
- Implemented the Linear Support Vector Machine.
- Implemented the RBF Support Vector Machine.
- Developed the six comparative classification approaches.

2. Proposed methodology:

- Implemented the combined HOG + PCA + RBF SVM approach.
- Integrated feature extraction, dimensionality reduction and classification.

3. Experimental evaluation:

- Implemented stratified 3-fold cross-validation.
- Evaluated the six classification methods.
- Calculated accuracy, precision, recall and Macro-F1.
- Compared execution times and performance variations.

4. Result analysis:

- Generated the model comparison tables.
- Analysed confusion matrices.
- Generated classification reports.
- Evaluated the final test-set predictions.
- Identified the best-performing model based on cross-validation results.

5. Documentation:

- Contributed to explaining the classification methods, experimental methodology, evaluation metrics, results, discussion and conclusion.

A.3 Shared Responsibilities

Both members jointly contributed to:

- Project planning and problem formulation.
- Understanding the overall pattern recognition workflow.
- Reviewing experimental results.
- Interpreting the final findings.
- Discussing project limitations and future improvements.
- Reviewing and finalising the project report.
- Preparing the project for academic submission.